Week 12 · Where the state goes: extraction and lifting

Technical Interaction Design · ITU · Autumn 2026

Built 12 November 2026 · commit `d9b3360`
Latest version: https://itu-tid.github.io/lecture-notes-pdf/Week-12.pdf

CONTENTS

Patterns of Component Communication

- Data goes down, as props
- Events come back up, via callbacks
- Passing `setTodos` down gives every row power over the whole list

Refactoring by Extracting Components

- Where the State Goes
- The same function, written twice

Updater functions for state

- Why would you use this
- The other way the setter surprises you: editing what is already in state

The `useRef` hook

- Read More

Custom Hooks

- Every page that loads something repeats the same twelve lines
- A custom hook is a function whose name starts with `use` and that calls other hooks
- A hook is worth writing even when only one component uses it
- The hook returns data, and the component decides what it looks like
- Each call gets its own state; a hook shares code, not data
- A fix made in the hook is made on every screen
- A polling timer belongs in a hook, because its cleanup is easy to forget
- A hook can wrap a browser event just as well as a fetch

Patterns of Component Communication

Source: Lectures/Technical/React/Patterns-of-Component-Communication.md — something unclear? Open an issue, or send a pull request.

Components are arranged in a tree, and information has to travel along it. There are only two directions, and they use different mechanisms.

You have been writing both since week 2, without the names. The to-do list hands each row a `todo` and an `onRemove`; the row calls `onRemove` and knows nothing about what happens next. This note is that arrangement, named, and the shortcut that looks easier and is not.

Data goes down, as props

The parent hands the child what it needs to draw itself: for a row, the `todo` it should display. Data goes down.

Something unclear in this section? Open an issue.

Events come back up, via callbacks

The child does not know who its parent is — and should not. Imagine a button: it has no idea who put it there, and it would be a worse button if it did.

What it does have is a function it was handed. Calling it is how the child says *something happened*, without knowing or caring what that means:

```
function TodoList() {
  const [todos, setTodos] = useState([]);

  function handleRemove(id) {
    setTodos(todos.filter((t) => t.id !== id));
  }

  return (
    <ul>
      {todos.map((todo) => (
        <TodoItem
          key={todo.id}
          todo={todo}          // down: what to draw
          onRemove={handleRemove}   // up: a way to report a click
        />
      ))}
    </ul>
  );
}

function TodoItem({ todo, onRemove }) {
  return (
    <li>
      {todo.text}
      <button onClick={() => onRemove(todo.id)}>×</button>
    </li>
  );
}
```

Read the two directions:

- **Down, as props.** `TodoList` hands each row its `todo` — the data it needs to draw.
- **Up, as a callback.** `TodoItem` is handed `onRemove` and calls it. It does not know what happens next: it is not deleting anything, it is *reporting a click*.

`TodoItem` knows nothing about `filter`, about the array, or even that a list exists. Drop it into an app that archives items instead of deleting them, passing the same `onRemove`, and it works unchanged. That is what you get for keeping the arrow pointing one way.

Something unclear in this section? Open an issue.

Passing `setTodos` down gives every row power over the whole list

The tempting shortcut is to pass `setTodos` down and let the row remove itself:

```
<TodoItem todo={todo} todos={todos} setTodos={setTodos} />   // do not do this
```

It works. It also means every row now knows the whole list, and the shape of it, and how deletion is implemented — so changing any of those means changing the row too. You have swapped one small prop for three, and gained a component that only works in this app.

Hand down what the child needs to know, and a way to tell you something happened. Nothing else.

Next: once you extract components, two of them end up needing the same state, and neither owns it. That is *lifting state up*, and it is in Refactoring by Extracting Components — because extraction is what creates the problem in the first place.

Something unclear in this section? Open an issue.

Exam Questions

1. What are the two directions information travels between components, and what carries each?

2. `TodoItem` has a delete button but cannot delete anything. Why is that the right arrangement?

3. What goes wrong if you pass `setTodos` to every row instead of an `onRemove` callback?

Refactoring by Extracting Components

Source: Lectures/Technical/Structure/Refactoring-by-Extracting-Components.md — something unclear? Open an issue, or send a pull request.

Lesson: refactoring is critical for having a codebase that is maintainable and pleasant to work with

Challenge: Compare the following equivalent versions of `Home` in todo-26

- They have **exactly the same behavior**: the same page, the same links and buttons
- Which of them is easier to read?

Version 1 — `git checkout week-08-service-layer`

```
<h2>My lists</h2>
{lists.map((item) => (
  <div key={item.id}>
    <Link to={`/lists/${item.id}`}>{item.get("name")}</Link>
    <AddForm
      onAdd={(username) => onShare(item, username)}
      buttonLabel="Share with (username)"
    />
  </div>
))}

{memberships.length > 0 && <h2>Shared with me</h2>}
{memberships.map((membership) => (
  <p key={membership.id}>
    <Link to={`/lists/${membership.get("list").id}`}>
      {membership.get("list").get("name")}
    </Link>{" "}
    shared by{" "}
    {membership.get("list").get("owner")?.get("username") ?? "someone"}{" "}
    <button onClick={() => onLeave(membership)}>Leave</button>
  </p>
))}

<h3>List Manager</h3>
<AddForm onAdd={onAddList} buttonLabel="New list" />
<h3>Account</h3>
<button onClick={onLogout}>Logout</button>
```

Version 2 — `git checkout week-08-components`

```
<h2>My lists</h2>
{lists.map((list) => (
  <MyListRow key={list.id} list={list} onShare={onShare} />
))}

{memberships.length > 0 && <h2>Shared with me</h2>}
{memberships.map((membership) => (
  <SharedListRow key={membership.id} membership={membership} onLeave={onLeave} />
))}

<h3>List Manager</h3>
<AddForm onAdd={onAddList} buttonLabel="New list" />
<h3>Account</h3>
<button onClick={onLogout}>Logout</button>
```

- Version 2 reads like the page: two headings, two kinds of row, a form, a button. Everything in it is at the same abstraction level, that of the page.
- Version 1 is at every level at once
  - the page: headings, sections
  - one row: a link, a form, a button
  - the data: `membership.get("list").get("owner")?.get("username")`, a pointer followed through a pointer
  - the text between the elements: `{" "}`

The details did not go away. They moved to `SharedListRow`, where they are the whole point, and you read them only when the row is what you care about.

Principle: Code in a component should be at the same abstraction level

Advice: iterate. iterate. iterate

- there is no way to arrive to simple code without a lot of iteration

Advice 2: first make it work. then refactor it and make it nice

- you never get it right from the first time

Where the State Goes

Extraction has a consequence. Split one component into three and some of them will need to agree about the same piece of state — which now belongs to none of them. The two patterns from Component Communication (props down, callbacks up) are not enough on their own.

sibling to sibling by "*lifting the state*"

Sometimes, you want the state of two components to always change together.

To do it, you *lift the state up*:

- remove state from both of them
- move it to their closest common parent, and then
- pass it down to them via props

This is a common thing you will do when writing React

Example: in the TODO list application, ensuring that one can't start two tasks in two different lists

Sending a message from one component to the other

- also has to be done via the parent
- in our case, imagine that you want to move one of the tasks, from one list to the other
- the parent would have to know about all the lists and a given list will have to call a callback on the parent to decide to which list does one

To read for next time: Sharing state between components

- talks about lifting state with an example

Exam Questions

1. What does "lifting state up" mean in React?

2. Why is this principle important: *"Code in a component should be at the same abstraction level"*?

The same function, written twice

Components are not the only thing worth extracting. A common case: `const getCurrentUser = async () ⇒ …` defined once in `ProfileBar` and again in `Settings`, because both screens needed it and neither knew about the other.

Two copies of a function are two things to fix when the backend changes, and they drift silently — one gets a null check, the other does not. Pull it into a **custom hook** (`useCurrentUser`) and both screens call the same code.

The signal to watch for: you are about to write something you have a feeling you already wrote. You probably did.

Something unclear in this section? Open an issue.

Updater functions for state

Source: Lectures/Technical/React/Updater-Functions-for-State.md — something unclear? Open an issue, or send a pull request.

> **You do not need this yet.** `setTodos([...todos, item])` is correct wherever you read the state and write it back in the same breath — adding, deleting, toggling. The updater form earns its keep only when the value you captured has gone stale: two setter calls in one handler, or a callback that fires later — a response arriving, a timer, a live query pushing a change.

State can be updated with either **updater functions** like in the first button below or by calling the setter function with an actual value, the second button below. One increments with three the value of the state var. The other with one.

```
export defult function Counter() {

  const [number, setNumber] = useState(0)

    return (
      <>
        <button onClick={() => {

          // updater function that takes a callback
          // all the ones below are the same
          // the name of the vaiable does not matter!
          setNumber(x => x + 1);
          setNumber(n => n + 1);
          setNumber(number => number + 1);
        }}>+3</button>

        <button onClick={() => {

          // direct update of variable
          // although you call it three times,  value is incremented with 1!
          setNumber(number + 1);
          setNumber(number + 1);
          setNumber(number + 1);
        }}>+1</button>
      </>
  }
```

Updater Function vs. **Calling with Actual Value**

- difference is between passing a callback and an actual value to the react state updater
- the updater will do this work at a later point, we don't control when that happens

Why would you use this

- Queueing A Series of State Updates
- When a lambda function risks capturing a stale state

Something unclear in this section? Open an issue.

The other way the setter surprises you: editing what is already in state

Since week 2 you have been building new arrays and new objects rather than editing the ones you had:

```
setTodos(todos.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
```

Here is what that habit was protecting you from. This is the version people write when they are in a hurry:

```
const copy = [...todos];                    // a new array...
copy.find((t) => t.id === id).done = true;  // ...holding the SAME objects
setTodos(copy);
```

The array is genuinely new, so React sees a change and re-renders, and the screen is right. **This code works**, which is the whole problem with it: nothing tells you off, so the habit survives.

What it did was reach into a to-do React was already holding and edit it where it lay. That costs you as soon as anything else is holding the same object:

- **Anything that kept the previous list** — an undo stack, or a comparison of before and after — is now holding a list whose contents changed underneath it. The past was rewritten.
- **Anything that skips work by comparing objects** — a memoised child, an effect with the item in its dependency array — sees the same object it saw last time and concludes nothing changed. The screen goes stale, and nothing looks wrong in your code.

You met this shape in week 2, with props: nothing physically stopped you pushing onto an array a parent had handed you, and it was wrong anyway. React is built on the assumption that what you put in state is never edited afterwards. So:

Treat everything already in state as read-only. To change it, build the new version and hand *that* to the setter — which is what `map` with a spread has been doing all along.

Something unclear in this section? Open an issue.

Exam Questions

1. What will happen when the button is clicked three times?

```
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
  }

  return <button onClick={handleClick}>{count}</button>;
}
```

2. What will be displayed after clicking the “+3” button?

```
function Counter() {
  const [number, setNumber] = useState(0);

  return (
    <button onClick={() => {
      setNumber(n => n + 1);
      setNumber(n => n + 1);
      setNumber(n => n + 1);
    }}>+3: {number}</button>
  );
}
```

3. When would you use an updater function vs. passing a direct value to setState?

4. This code gives the setter a brand-new array, the screen updates, and it is still wrong. Why?

```
const copy = [...todos];
copy.find((t) => t.id === id).done = true;
setTodos(copy);
```

The `useRef` hook

Source: Lectures/Technical/React/The-useRef-Hook.md — something unclear? Open an issue, or send a pull request.

Allows you to get hold of a DOM element and manipulate it

Example: Automatically focusing a given element on first render

```
import {useRef, useEffect} from 'react'

const InputRefExample = () => {

      const inputRef = useRef(null);

      useEffect(() => {
        inputRef.current?.focus()
      }, [])

      return(
        <div>
          <input ref={inputRef} />
          <button>
            Submit
          </button>
        </div>
      )
    }

export default inputRefExample
```

Canonical use of the `useRef` hook

1. Import `useRef`
2. Declare a ref object (i.e. `inputRef`) - initialize it with null, it's fine
3. Pass your object as the `ref` attribute to the desired DOM element
4. Refer to the DOM object by using the `current` property

Other Examples of manipulating the DOM

- scrolling to a given node
- getting the dimensions of a given DOM node
- keeping track of a timer (see the TODO list application)

Notes

Optional chaining to prevent errors

The optional chaining (`?.`) prevents errors if the element isn't mounted yet.

```
      useEffect(() => {
        inputRef.current?.focus()
      }, [])
```

The reference is persisted between renders

State changes of the referenced element are not observed by React

Read More

- useRef in the react documentation
- Nice example of controlling a video player component
  - you need to interact with the video element in the DOM
  - the DOM has already a powerful implementation of
  - get a reference to the relevant DOM element
  - call the `start` and `stop` methods

Exam Questions

1. What is the purpose of the useRef hook?

2. What does this code do and why use optional chaining?

```
const inputRef = useRef(null);

useEffect(() => {
  inputRef.current?.focus();
}, []);

return <input ref={inputRef} />;
```

Something unclear in this section? Open an issue.

Custom Hooks

Source: Lectures/Technical/React/Custom-Hooks.md — something unclear? Open an issue, or send a pull request.

The Three States of Remote Data ended with a promise: every screen that fetches needs the same `useState` pair and the same `useEffect`, and "later in the course we come back and do something about this one." This is that note.

Every page that loads something repeats the same twelve lines

Here is `ListPage` from todo-26, once its query has moved into `loadList(listId)` in a service, as in the week 8 exercise:

```
export default function ListPage() {
  const { listId } = useParams();

  const [list, setList] = useState(null);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function load() {
      try {
        setList(await loadList(listId));
      } catch (err) {
        setError(err.message);
      }
    }
    load();
  }, [listId]);

  if (error) return <p>{error}</p>;
  if (!list) return <p>Loading…</p>;
  return <ToDoList list={list} />;
}
```

The home page that loads your lists has the same two `useState` calls, the same `useEffect`, and the same `try` / `catch`. So does every page in your group project that shows something from Parse. Only two things change from one copy to the next: *which* service function is called, and *what* the page renders.

The service layer could not remove this duplication, because it is not about talking to Parse. It is about **React state**: something is loading, then it is there or it failed. Moving state out of a

component needs a hook.

Something unclear in this section? Open an issue.

A custom hook is a function whose name starts with `use` and that calls other hooks

That is all it is. There is no API to register it and nothing to import from React to make one:

```
// src/hooks/useList.js
import { useEffect, useState } from "react";
import { loadList } from "../services/listService.js";

export function useList(listId) {
  const [list, setList] = useState(null);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function load() {
      try {
        setList(await loadList(listId));
      } catch (err) {
        setError(err.message);
      }
    }
    load();
  }, [listId]);

  return { list, error };
}
```

And the page shrinks to what it is actually about:

```
export default function ListPage() {
  const { listId } = useParams();
  const { list, error } = useList(listId);

  if (error) return <p>{error}</p>;
  if (!list) return <p>Loading…</p>;
  return <ToDoList list={list} />;
}
```

The `use` prefix is not decoration. It tells the rules of hooks linter that this function calls hooks, so it checks the callers too: no `useList` inside an `if`, a loop, or an event handler. Name it `getList` and the linter goes quiet, but the rules still apply, and breaking them still breaks your component.

Something unclear in this section? Open an issue.

A hook is worth writing even when only one component uses it

Reuse is the reason that is easiest to see, and not the most important one. Two others hold even for a hook with a single caller:

- **Decomposition.** A component that loads a list, polls it, tracks whether the app is online and handles the install prompt does four things, and reads like it. Each hook pulled out of it takes one of those things away and gives it a name. What is left reads like a description of the screen: `useList(listId)`, `useInterval(reload, 10_000)`. The same instinct that splits a big component into smaller components (Extracting Components) applies to the logic inside one.
- **Encapsulation.** `ListPage` no longer knows that the list comes from Parse, that a stale answer has to be ignored, or that loading can fail in two ways. If you later replace polling with push, or Parse with something else, you change `useList` and no screen notices. A hook is a wall around a decision you might want to change.

So the question is not only “will I need this twice?” but “would this component be easier to read if this part had a name?”

Something unclear in this section? Open an issue.

The hook returns data, and the component decides what it looks like

A custom hook returns values (state, and functions that change it), never JSX. `useList` does not know whether the list ends up in a `<ul>`, a MUI `Card`, or a count in the navigation bar. That is the same split as the service layer, one level up:

layer	knows about	does not know about
service (`loadList`)	Parse, class names, field names	React, the screen
hook (`useList`)	React state, loading and failing	Parse, the screen
component (`ListPage`)	what the user sees	Parse, how loading works

When a hook starts returning `<p>Loading…</p>`, it has turned into a component with an odd name. Turn it into a component.

Something unclear in this section? Open an issue.

Each call gets its own state; a hook shares code, not data

If the home page and the navigation bar both call `useCurrentUser()`, each gets its **own** `useState`, inside its own component, and each runs its own `useEffect`. Two components, two requests, two copies of the user that can drift apart.

This is the most common misunderstanding about custom hooks. A hook is a way of reusing *logic*. When two components need the *same value*, that value lives in one place, in their common parent or in a context, and travels down as props. That is the topic of Patterns of Component Communication.

Something unclear in this section? Open an issue.

A fix made in the hook is made on every screen

There is a bug in the `useEffect` above, and in every copy of it in your project. Open list A, then quickly click to list B. Two requests are now on their way. If A's answer arrives *after* B's, the page shows list A under list B's URL.

The fix is to ignore an answer that arrives after the effect has been cleaned up:

```
useEffect(() => {
  let ignore = false;

  async function load() {
    try {
      const result = await loadList(listId);
      if (!ignore) setList(result);
    } catch (err) {
      if (!ignore) setError(err.message);
    }
  }
  load();

  return () => {
    ignore = true;   // listId changed, or the page closed: this answer is stale
  };
}, [listId]);
```

Written into twelve components, that fix gets forgotten in at least one of them. Written into `useList`, it is made once and every screen has it. This, more than the lines saved, is why you extract a hook: the tricky part of the pattern gets one home.

Something unclear in this section? Open an issue.

A polling timer belongs in a hook, because its cleanup is easy to forget

Efficient Communication shows how to reload data every few seconds with `setInterval`, and why the interval must be cleared when the component goes away. Forgetting that cleanup is exactly the kind of mistake a hook prevents. The first version is short:

```
// src/hooks/useInterval.js
import { useEffect } from "react";

export function useInterval(callback, milliseconds) {
  useEffect(() => {
    const id = setInterval(callback, milliseconds);
    return () => clearInterval(id);
  }, [callback, milliseconds]);
}
```

```
// the shopping list reloads every ten seconds while it is open
useInterval(reload, 10_000);
```

A ref keeps the latest callback without restarting the timer

That version has a problem you only see in use. `reload` is defined inside the component, so it is a **new function on every render**. It is in the dependency array, so every render clears the timer and starts a new one. Type into a form on that page, and each keystroke restarts the ten seconds: the list never reloads while you type.

Take `callback` out of the dependencies, and the opposite happens: the timer keeps the `reload` from the first render forever, with the state of the first render in it (a *stale closure*).

What the hook needs is a place to keep **the latest callback, without restarting the timer when it changes**: a value that survives renders, and whose change does not cause one. That is exactly what a ref is (The useRef Hook):

```
import { useEffect, useRef } from "react";

export function useInterval(callback, milliseconds) {
  const latestCallback = useRef(callback);

  // after every render, remember the newest callback; the timer is not touched
  useEffect(() => {
    latestCallback.current = callback;
  });

  // the timer starts once, and always calls whatever callback is newest
  useEffect(() => {
    const id = setInterval(() => latestCallback.current(), milliseconds);
    return () => clearInterval(id);
  }, [milliseconds]);
}
```

The component still writes `useInterval(reload, 10_000)`. All of this is inside the hook, which is the point: the tricky part has one home.

Something unclear in this section? Open an issue.

A hook can wrap a browser event just as well as a fetch

Hooks are not only for remote data. Anything a component has to *subscribe to and clean up after* is a candidate: a timer, the window's width, whether the browser is online.

Once a project has a `src/hooks/` folder, the hook names read like a list of what the app does: `useList`, `useInterval`. Each one names a single behavior, which is the same test you apply to a component name.

Something unclear in this section? Open an issue.

Exam Questions

1. What makes a function a custom hook? What changes if you rename `useList` to `getList`?

2. Two components both call `useCurrentUser()`. How many requests go to the backend, and how many copies of the user exist? What would you do if they must always show the same user?

3. Why should a custom hook return data rather than JSX?

4. Explain the bug that the `ignore` flag fixes. Why is a custom hook a better place for that fix than each component?

5. Write `useInterval(callback, milliseconds)`. What happens if you leave out the function that `useEffect` returns?

6. Give two reasons to extract a custom hook that only one component uses.

7. Why does `useInterval` keep the callback in a ref instead of putting it in the effect's dependency array?

References

- Reusing Logic with Custom Hooks, react.dev
- Fetching data, react.dev: the `ignore` flag